

CY SafeCampus

Analyse éthique de l'optimisation de l'évacuation d'urgence
Projet Génie Logiciel

ING1 - GI2 - C
2025-2026

Réalisé par : ABDOU Malak - AMAINI Maellys - BARHILI Samia - BENAMEUR Bayane

SOMMAIRE

Introduction.....	3
I) Présentation de l'équipe et du sujet.....	4
a) Présentation de l'équipe.....	4
b) Présentation du projet CY SafeCampus.....	4
c) Problématique et objectifs du projet.....	4
II) Problèmes rencontrés, solutions apportées et résultats obtenus.....	5
a) Problèmes d'organisation et de communication.....	5
b) Problèmes techniques rencontrés.....	5
c) Solutions mises en place.....	5
d) Résultats obtenus.....	6
III) Organisation de l'équipe.....	7
a) Répartition des tâches.....	7
b) Outils utilisés.....	7
c) Évolution de notre méthode de travail.....	7
IV) Flux de travail suivi.....	8
a) Analyse du cahier des charges.....	8
b) Conception du graphe et des classes principales.....	8
c) Développement et intégration.....	8
d) Tests et corrections.....	8
e) Finalisation.....	8
V) Conception technique et cas d'utilisation.....	9
a) Architecture générale.....	9
b) Diagramme de classes UML.....	9
c) Description rapide de l'architecture.....	10
d) Acteurs et cas d'utilisation principaux.....	10
e) Scénario d'évacuation d'urgence.....	11
Conclusion.....	12

Introduction

Dans le cadre du projet de Génie Logiciel, notre équipe a développé CY SafeCampus, une application JavaFX de simulation d'évacuation d'urgence. Le sujet imposait de travailler sur le déplacement d'agents dans un graphe avec une limitation du nombre d'agents par arête. Nous avons choisi de l'appliquer à un contexte concret : l'évacuation d'un bâtiment universitaire.

L'objectif n'est pas de remplacer un vrai dispositif de sécurité, mais de proposer une simulation permettant de représenter un bâtiment, d'observer les déplacements d'agents, de gérer la congestion et de tester différents itinéraires vers les sorties. Le projet final permet notamment de visualiser un graphe du bâtiment, de lancer une simulation, de déclencher une alerte, de recalculer des chemins et de sauvegarder ou recharger un état de simulation.

I) Présentation de l'équipe et du sujet

a) Présentation de l'équipe

Notre groupe est composé de quatre étudiantes : Malak Abdou, Maellys Amaini, Samia Barhili et Bayane Benameur. Le projet a été réalisé en groupe, avec une répartition progressive des tâches selon les besoins : modélisation du graphe, déplacement des agents, interface JavaFX, gestion des capteurs, tests, UML et rédaction du rapport.

b) Présentation du projet CY SafeCampus

CY SafeCampus est une application de simulation qui représente un bâtiment sous forme de graphe. Les salles, couloirs, escaliers et sorties sont modélisés par des classes dédiées. Les agents représentent les personnes présentes dans le bâtiment et se déplacent progressivement dans le graphe en fonction de leur destination, de leur vitesse, de leur comportement et de l'état des passages.

L'application comprend une interface administrateur permettant de visualiser et modifier le graphe, d'ajouter des agents, de lancer ou pauser la simulation, de déclencher une alerte et de surveiller l'évolution de l'évacuation. Une vue superviseur permet aussi de suivre une salle donnée et les consignes associées.

c) Problématique et objectifs du projet

La problématique principale est la suivante : comment simuler une évacuation dans un bâtiment en tenant compte à la fois du chemin à suivre et des limites de capacité des passages ? Dans une évacuation réelle, le chemin le plus court n'est pas toujours le meilleur si un couloir est saturé, bloqué ou ralenti. Le projet devait donc prendre en compte la structure du bâtiment, les capacités, les agents et les changements de situation pendant la simulation.

Les objectifs retenus pour notre version finale sont les suivants :

- modéliser le bâtiment avec des éléments reliés entre eux par des portes et passages ;
- limiter le nombre d'agents dans certains éléments afin de représenter la congestion ;
- calculer des chemins d'évacuation avec un algorithme de type Dijkstra ;
- permettre un recalcul des routes lorsque la situation change ;
- visualiser la simulation avec une interface graphique JavaFX ;
- gérer des capteurs, une alerte et la sauvegarde de la simulation.

II) Problèmes rencontrés, solutions apportées et résultats obtenus

a) Problèmes d'organisation et de communication

Au début du projet, la première difficulté a été de trouver des créneaux communs pour travailler efficacement. Les emplois du temps étaient différents et il n'était pas toujours possible de réunir tout le groupe. Nous avons donc avancé par tâches séparées, puis nous avons organisé des points réguliers pour intégrer le travail de chacune.

La communication a aussi demandé une adaptation. Nous utilisons plusieurs outils en même temps : WhatsApp pour échanger rapidement, Google Docs pour le rapport et GitHub pour partager le code. Cette organisation a parfois provoqué une dispersion des informations. Pour corriger cela, nous avons progressivement clarifié les tâches à faire et nous avons davantage utilisé GitHub comme référence pour le code final.

b) Problèmes techniques rencontrés

La première difficulté technique importante a été la modélisation du bâtiment. Nous pensions d'abord représenter les couloirs comme de simples arêtes entre les salles. Cette solution était trop limitée, car elle ne permettait pas de gérer correctement la capacité, la congestion, la distance ou la présence d'agents dans les couloirs. Nous avons donc fait évoluer le modèle : les couloirs, halls et escaliers sont devenus des objets de type Passage, reliés aux salles par des Door.

Le calcul de chemin a également posé problème. Un simple plus court chemin ne suffisait pas, car il ne prenait pas en compte la vitesse des agents, la saturation des passages ou les zones bloquées. Nous avons donc distingué deux modes : un chemin le plus court basé sur la distance et un chemin le plus rapide basé sur le temps estimé, la vitesse de l'agent, le facteur de vitesse du passage et la congestion.

La simulation en temps réel a demandé plusieurs ajustements. Les agents ne devaient pas se téléporter d'un nœud à l'autre : ils devaient progresser étape par étape, attendre lorsque certains passages étaient saturés et changer de route en cas d'alerte. Il fallait aussi éviter les incohérences lorsque plusieurs agents modifiaient l'occupation d'un même passage au même moment.

Enfin, l'intégration avec JavaFX a demandé de bien séparer le modèle logique et l'affichage. Les actions de l'interface, comme ajouter un nœud, supprimer une connexion, déplacer un agent ou déclencher une alerte, devaient modifier correctement le graphe utilisé par la simulation.

c) Solutions mises en place

Pour résoudre ces problèmes, nous avons mis en place une architecture plus claire. Le modèle est centré sur la classe Graph, qui contient les éléments du bâtiment, les agents, les capteurs et les observateurs. Les éléments physiques héritent de BuildingElement, ce qui permet de manipuler les salles, passages et sorties de manière cohérente.

Nous avons utilisé plusieurs patrons de conception. Le patron Strategy permet de changer le comportement de déplacement des agents avec EvacuateStrategy, PanicStrategy et GuideStrategy. Le patron Observer est utilisé pour notifier les agents lors d'une alerte et pour permettre aux capteurs

d'envoyer des événements à l'administrateur. L'organisation générale suit aussi une logique MVC : GraphController fait le lien entre le modèle et les vues JavaFX.

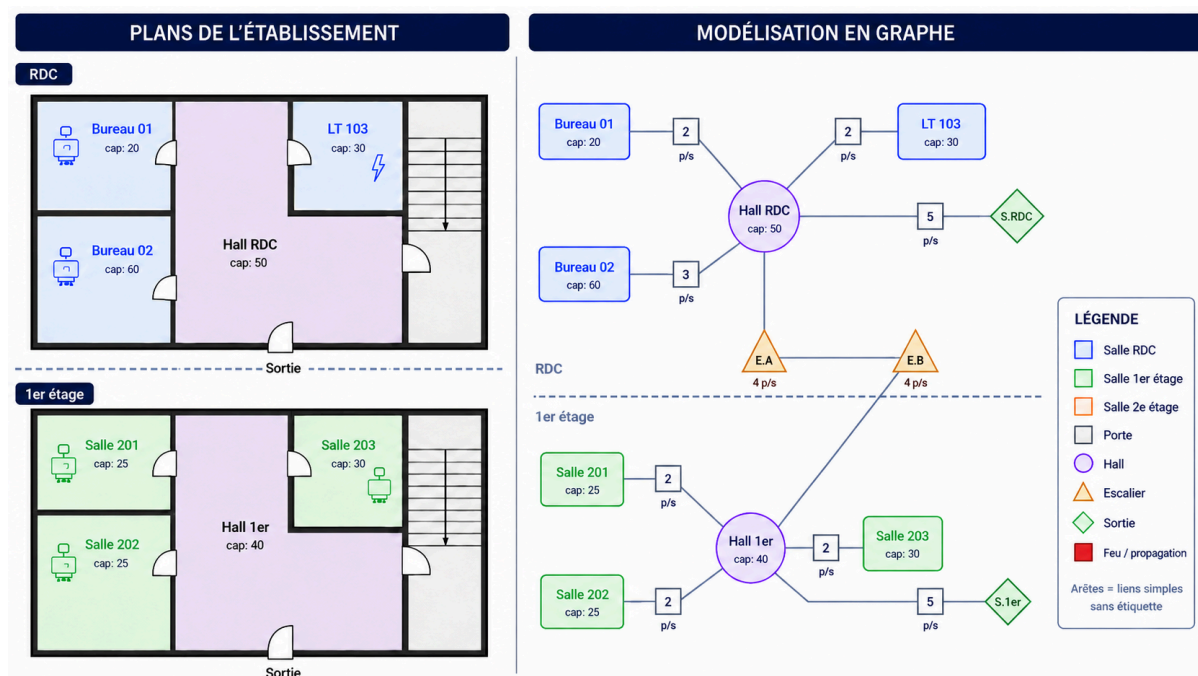
Pour la simulation, une classe SimulationEngine exécute les ticks de manière régulière. À chaque étape, les agents avancent, les capteurs peuvent détecter des événements et l'interface peut être rafraîchie. Pour conserver le travail, une classe SimulationSerializer permet de sauvegarder et recharger l'état d'une simulation.

d) Résultats obtenus

La version finale permet de représenter un bâtiment sous forme de graphe, d'ajouter ou modifier des éléments, de placer des agents, de lancer la simulation et de déclencher une alerte. Les agents peuvent suivre un chemin vers une sortie, être ralentis par la congestion et être redirigés lorsque certaines zones deviennent bloquées.

L'application contient aussi des capteurs de présence et de fumée, une interface administrateur, une interface superviseur, un moteur de simulation, un calculateur de chemins et un système de sauvegarde/chargement. Le résultat final répond donc au thème du projet : déplacement d'agents dans un graphe avec prise en compte de capacités limitées.

Certaines limites restent présentes : l'application est une simulation pédagogique et non un système de sécurité réel. Les tests ont surtout été réalisés manuellement à partir de scénarios d'évacuation, et certains rôles existent davantage au niveau du modèle que sous forme d'interfaces complètes. Ces limites sont normales pour un projet de première année, mais elles indiquent aussi des pistes d'amélioration possibles.



III) Organisation de l'équipe

a) Répartition des tâches

La répartition du travail n'a pas été totalement figée dès le départ. Elle a évolué selon l'avancement du projet et les difficultés rencontrées. Nous pouvons cependant distinguer plusieurs grands pôles de travail :

- conception du modèle du bâtiment : BuildingElement, Room, Passage, Exit et Door ;
- développement de la logique de déplacement : Agent, PathFinder et stratégies ;
- gestion des capteurs et des alertes : Sensor, SmokeSensor, PresenceSensor et SensorEvent ;
- interface graphique : LoginView, AdminView et SupervisorView ;
- tests, intégration, diagrammes UML et rédaction du rapport.

b) Outils utilisés

Nous avons utilisé Java et JavaFX pour le développement, Maven pour la configuration du projet, GitHub pour le partage du code et Google Docs pour la rédaction. Les échanges rapides se faisaient principalement par messagerie. Cette combinaison d'outils nous a permis d'avancer même lorsque tout le groupe ne pouvait pas être disponible en même temps.

c) Évolution de notre méthode de travail

Au début, nous avons parfois travaillé de manière trop indépendante, ce qui a créé des problèmes lors de l'intégration. Par la suite, nous avons mieux identifié les fichiers sensibles, notamment les classes centrales du modèle et le contrôleur. Nous avons également pris l'habitude de vérifier plus régulièrement que les changements dans une partie du projet ne cassaient pas les autres parties.

Cette évolution a été importante, car le projet contient des classes très liées : une modification dans le graphe peut avoir un impact sur le déplacement des agents, l'affichage ou les capteurs. Le travail en groupe nous a donc appris à mieux communiquer avant de modifier les classes centrales.

IV) Flux de travail suivi

a) Analyse du cahier des charges

Nous avons commencé par analyser le thème du projet : déplacement d'agents dans un graphe avec limitation du nombre d'agents par arête. Nous avons choisi de transformer ce thème en une simulation d'évacuation, car ce contexte permettait de donner du sens aux notions de chemin, capacité, congestion et redirection.

b) Conception du graphe et des classes principales

La première étape technique a été de définir les classes principales. Le bâtiment est représenté par un Graph composé d'éléments de type BuildingElement. Les salles et sorties sont reliées aux passages par des portes. Cette modélisation a été choisie pour mieux gérer les capacités et éviter de réduire les couloirs à de simples liens abstraits.

c) Développement et intégration

Nous avons ensuite développé progressivement les fonctionnalités : création du graphe, ajout des agents, calcul de chemins, simulation par ticks, capteurs, alertes, puis interface graphique. Le contrôleur a été renforcé au fur et à mesure afin de relier les actions de l'interface aux données du modèle.

d) Tests et corrections

Les tests ont été réalisés à partir de scénarios simples : déplacement d'un agent vers une sortie, ajout de plusieurs agents dans un même passage, déclenchement d'une alerte, blocage d'une zone, sauvegarde puis rechargement d'une simulation. Ces tests ont permis de corriger des problèmes de chemin, d'occupation et de cohérence entre le modèle et l'affichage.

e) Finalisation

La dernière étape a consisté à nettoyer le projet, compléter le README, produire les diagrammes UML et rédiger le rapport final. Le rapport met l'accent sur les choix de conception, les difficultés rencontrées et les fonctionnalités réellement présentes dans la version finale.

V) Conception technique et cas d'utilisation

a) Architecture générale

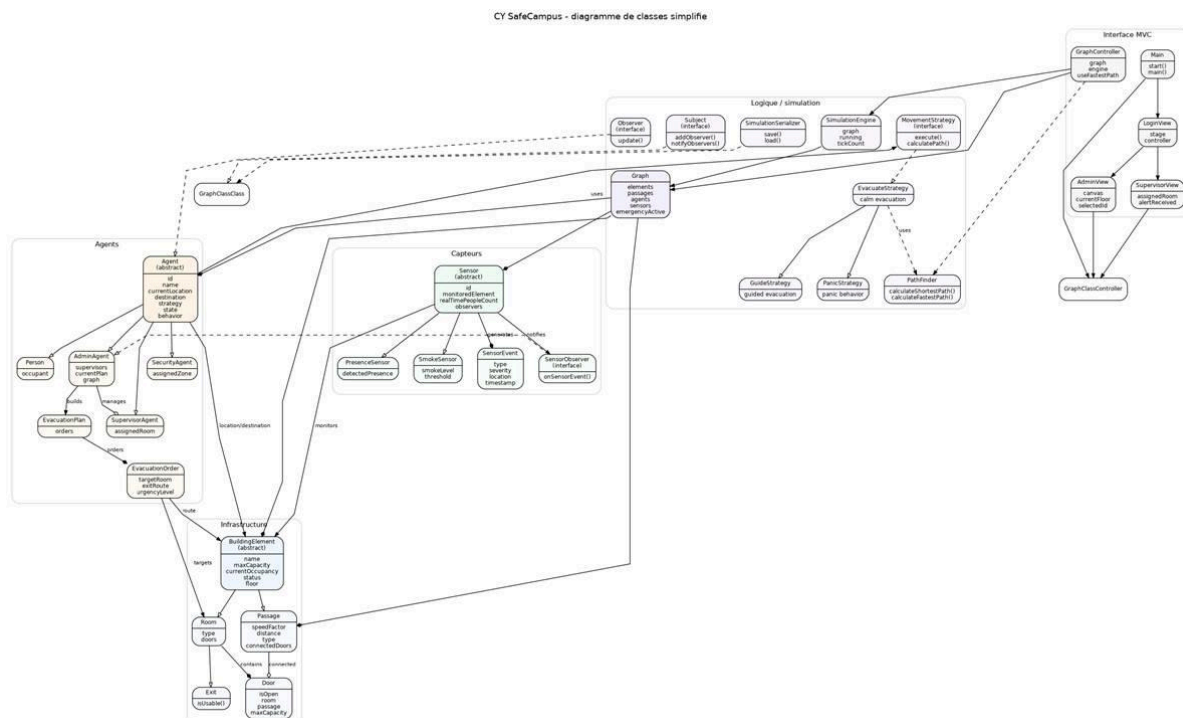
L'application est organisée autour de trois grands ensembles : le modèle, le contrôleur et les vues. Le modèle contient les données et la logique métier : bâtiment, agents, capteurs, graphe, simulation et calcul de chemin. Le contrôleur reçoit les actions de l'utilisateur et applique les changements au modèle. Les vues JavaFX affichent le bâtiment, les agents, les alertes et les informations utiles aux différents rôles.

Partie	Classes principales	Rôle
Modèle du bâtiment	BuildingElement, Room, Passage, Exit, Door	Représenter les espaces physiques et leurs capacités.
Agents	Agent, Person, AdminAgent, SupervisorAgent	Représenter les personnes et leurs comportements.
Déplacement	PathFinder, MovementStrategy, EvacuateStrategy, PanicStrategy, GuideStrategy	Calculer et exécuter les routes d'évacuation.
Simulation	Graph, SimulationEngine, SimulationSerializer	Faire évoluer la simulation et sauvegarder son état.
Capteurs	Sensor, PresenceSensor, SmokeSensor, SensorEvent	Détecter des événements et générer des alertes.
Interface	GraphController, LoginView, AdminView, SupervisorView	Relier le modèle aux écrans JavaFX.

b) Diagramme de classes UML

Le diagramme ci-dessous présente une version lisible du diagramme de classes. Les méthodes ont été simplifiées pour garder une vue claire de l'architecture. Les relations principales montrent l'héritage, la composition, les associations et les dépendances entre les classes du projet.

Diagramme de classes UML simplifié



c) Description rapide de l'architecture

La classe Graph sert de conteneur principal. Elle stocke les éléments du bâtiment, les agents, les capteurs et les observateurs. Les agents sont notifiés lorsqu'une alerte est déclenchée, puis leur stratégie peut adapter leur déplacement.

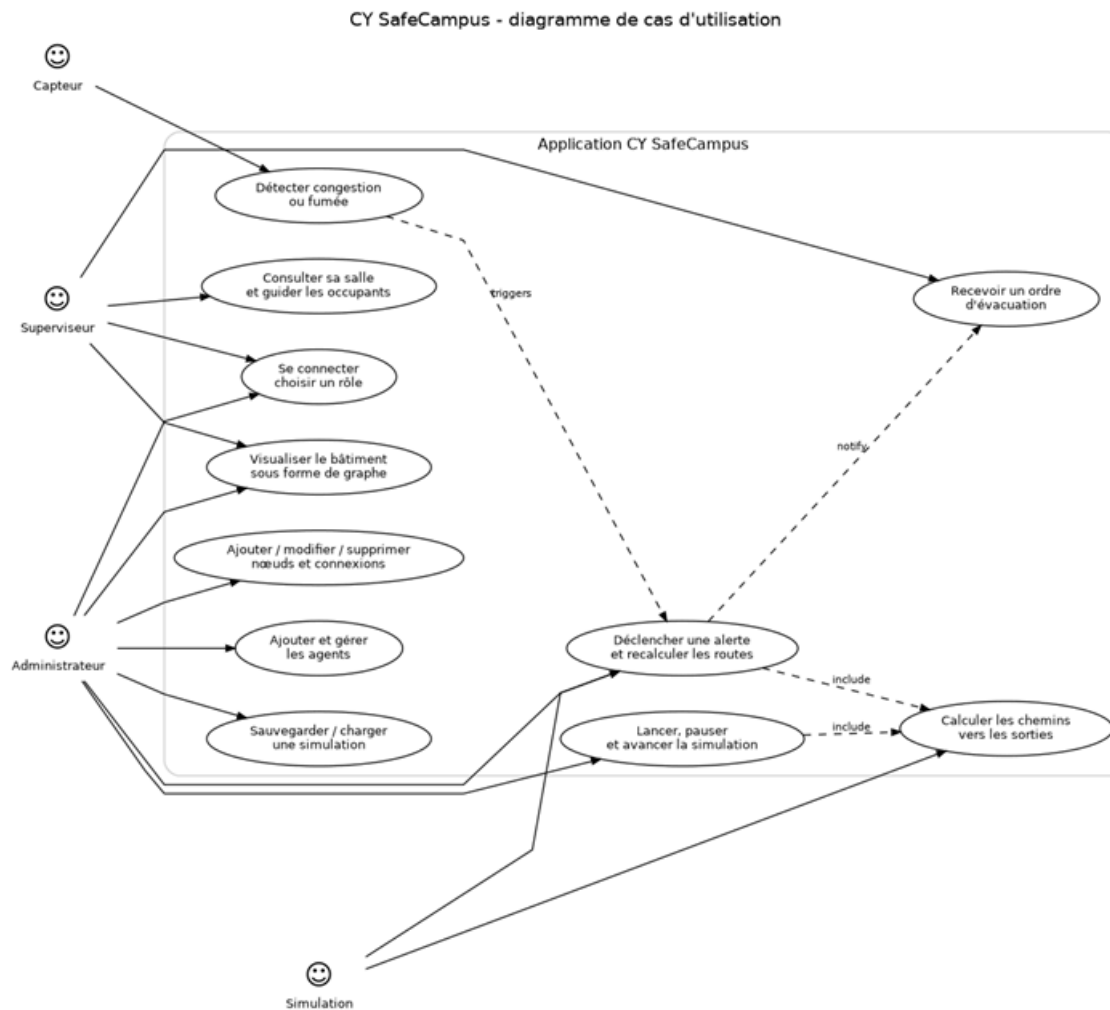
PathFinder centralise le calcul des routes. Il propose un mode distance et un mode temps. Dans le mode temps, la congestion et la vitesse des agents influencent le coût du passage. Cette différence est importante, car elle permet de simuler des itinéraires plus réalistes qu'un simple plus court chemin.

Les capteurs suivent un modèle séparé : ils surveillent un élément du bâtiment, créent un SensorEvent et notifient les observateurs concernés. L'administrateur peut ensuite recevoir l'événement et produire un plan ou des ordres d'évacuation.

d) Acteurs et cas d'utilisation principaux

Les principaux acteurs du système sont l'administrateur, le superviseur, les capteurs et la simulation elle-même. Les occupants sont représentés par les agents dans le modèle. Les cas d'utilisation les plus importants sont les suivants :

- l'administrateur visualise le bâtiment, modifie le graphe, ajoute des agents, déclenche une alerte et sauvegarde la simulation ;
- le superviseur consulte sa salle, reçoit les informations d'évacuation et guide les occupants ;
- les capteurs détectent la présence, la fumée ou une situation de surcharge ;
- la simulation calcule les routes, fait avancer les agents et met à jour l'état du graphe.

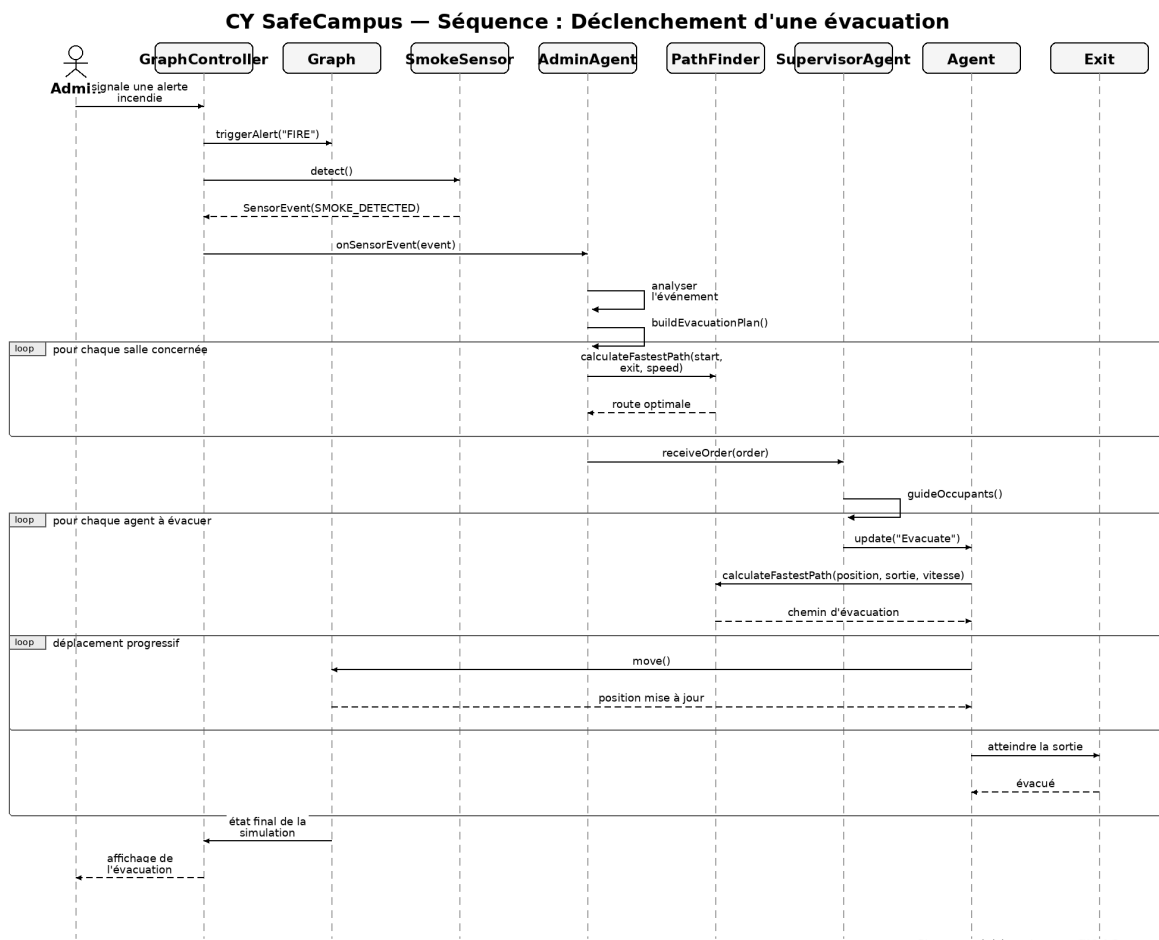


e) Scénario d'évacuation d'urgence

Un scénario typique commence par la présence d'agents dans plusieurs salles. L'administrateur lance la simulation. Lorsqu'une alerte est déclenchée, certaines zones peuvent devenir bloquées ou moins attractives. Les agents concernés reçoivent une nouvelle destination et le système recalcule leur route vers une sortie. Si un passage est saturé, certains agents peuvent attendre ou emprunter un autre chemin selon le mode de calcul choisi.

Ce scénario permet de vérifier que la simulation respecte bien le principe du sujet : les agents ne se déplacent pas simplement sur un graphe théorique, mais dans un environnement où la capacité, les portes, la congestion et les événements influencent leur progression.

Le diagramme de séquence suivant illustre ce scénario d'évacuation. Il détaille les échanges entre l'administrateur, le contrôleur, le graphe, les capteurs, l'agent administrateur, les superviseurs et les agents à évacuer.



Ce diagramme montre que l'évacuation ne repose pas uniquement sur une action manuelle. L'alerte déclenche une série d'interactions entre les capteurs, le graphe et les agents. Le système peut ainsi recalculer les chemins et adapter les déplacements selon l'état du bâtiment.

Conclusion

CY SafeCampus nous a permis de mettre en pratique plusieurs notions importantes de génie logiciel : modélisation objet, graphe, algorithme de chemin, patrons de conception, interface graphique et travail en équipe. Le projet a évolué au fur et à mesure de notre compréhension du sujet. La version finale est plus cohérente que les premières versions, notamment grâce à la séparation entre salles, passages et portes, ainsi qu'à l'ajout de la simulation par ticks et des capteurs.

Le projet répond au cahier des charges en proposant une application Java qui simule le déplacement d'agents dans un graphe avec des capacités limitées. Il reste perfectible, notamment sur l'automatisation des tests, la précision physique de la simulation et le développement d'interfaces complètes pour tous les rôles. Malgré ces limites, l'application obtenue est fonctionnelle et montre clairement l'intérêt d'une approche dynamique pour étudier les flux d'évacuation.